Kim Fischer Competition Report S26

Saemin Kim and Matthew Fischer

0. Github Repo Link:

https://github.com/matthewtfischer/KimFischer_ENV797_TSA_ForecastCompetition_S26.git

1. Data Wrangling and Vanila Model

Setting R code chunk options

First R code chunk is used for setting the options for all R code chunks. The choice echo=TRUE means both code and output will appear on report, include = FALSE neither code nor output is printed.

Loading packages and initializing

Second R code chunk is for loading packages. By setting message = FALSE, the code will appear but not the output.

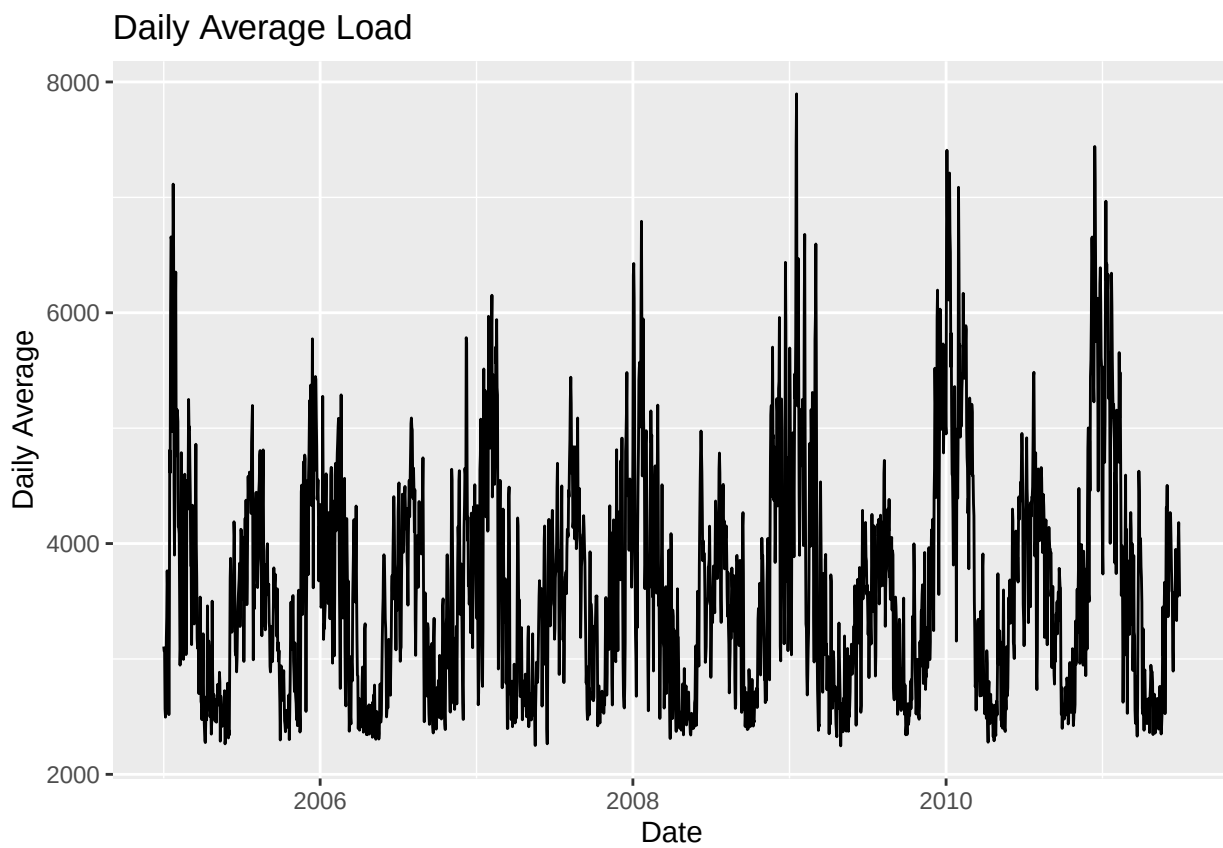
```
library(lubridate)
library(ggplot2)
library(forecast)
#library(Kendall)
library(tseries)
#library(outliers)
library(tidyverse)
library(smooth)

library(kableExtra)
library(readxl)
library(openxlsx)
library(prophet)
```

Importing data

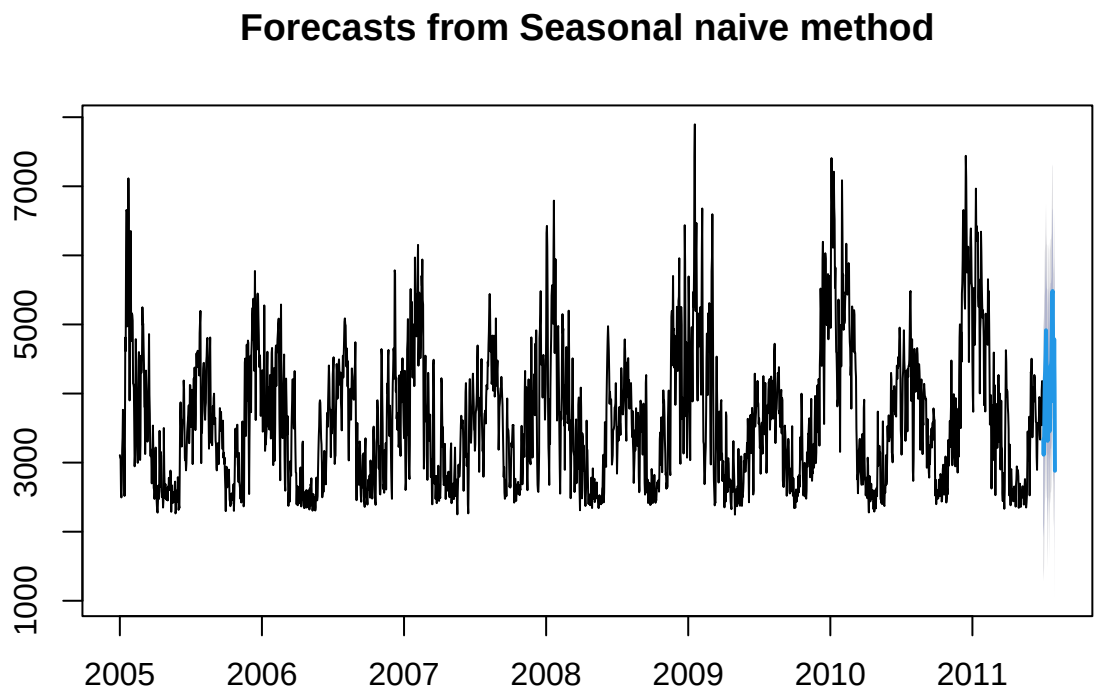
```
data <- read_excel("load.xlsx")

daily_data <- data %>%
  mutate(
    date = as.Date(date),
    Year = year(date),
    Month = month(date),
    Day = day(date)
  ) %>%

  rowwise() %>%
```

```r
  mutate(
    daily_avg = mean(c_across(h1:h24), na.rm = TRUE)
  ) %>%
  ungroup() %>%

  select(meter_id, date, Year, Month, Day, daily_avg)

daily_data <- daily_data %>%
  filter(!is.na(daily_avg), !is.nan(daily_avg), is.finite(daily_avg))
```

```r
ggplot(daily_data, aes(x = date, y = daily_avg)) +
  geom_line() +
  labs(title = "Daily Average Load", y = "Daily Average", x = "Date")
```




```r
data_ts <- msts(daily_data$daily_avg,
                            seasonal.periods =c(7, 365),
                            start=c(2005,01))

SNAIVE_data <- snaive(data_ts, h=31)
plot(SNAIVE_data)
```




```
forecast_values <- as.numeric(SNAIVE_data$mean)
```

```
submission <- read.csv("submission_template.csv")

submission$load <- forecast_values
write.csv(submission, "seasonal_naive_model.csv", row.names = FALSE)
```

2. External Variables and Train/Test Split

Temperature, Humidity Data Wrangling

```
temp <- read_excel("temperature.xlsx")
humid <- read_excel("relative_humidity.xlsx")

# daliy avg of temp
temp_daily <- temp %>%
  mutate(date = as.Date(date)) %>%
  rowwise() %>%
  # hourly avg of all t_ws1 ~ t_ws28
  mutate(hourly_temp_avg = mean(c_across(starts_with("t_ws")), na.rm = TRUE)) %>%
  ungroup() %>%
  group_by(date) %>%
  # daily avg
  summarise(daily_temp = mean(hourly_temp_avg, na.rm = TRUE))
```

```r
# daliy avg of humid
humid_daily <- humid %>%
  mutate(date = as.Date(date)) %>%
  rowwise() %>%
  # hourly avg of all rh_ws1 ~ rh_ws28
  mutate(hourly_humid_avg = mean(c_across(starts_with("rh_ws")), na.rm = TRUE)) %>%
  ungroup() %>%
  group_by(date) %>%
  # daily avg
  summarise(daily_humid = mean(hourly_humid_avg, na.rm = TRUE))

# Join the data
daily_data <- daily_data %>%
  left_join(temp_daily, by = "date") %>%
  left_join(humid_daily, by = "date")

head(daily_data)
```

```
## # A tibble: 6 x 8
##   meter_id date        Year Month   Day daily_avg daily_temp daily_humid
##   <chr>    <date>     <dbl> <dbl> <int>     <dbl>      <dbl>       <dbl>
## 1 0072     2005-01-01  2005     1     1     3108.       53.6        76.7
## 2 0072     2005-01-02  2005     1     2     3068.       53.8        80.5
## 3 0072     2005-01-03  2005     1     3     3061.       55.9        81.2
## 4 0072     2005-01-04  2005     1     4     2566.       61.7        74.8
## 5 0072     2005-01-05  2005     1     5     2496.       60.4        76.1
## 6 0072     2005-01-06  2005     1     6     2735.       62.0        78.0
```

Train and Validation Split

```r
# Training = 2005-01-01 to 2010-06-30 / Validation = 2010-07-01 to 2010-07-31
train_data <- daily_data %>% filter(date >= as.Date("2005-01-01") & date <= as.Date("2010-06-30"))
valid_data <- daily_data %>% filter(date >= as.Date("2010-07-01") & date <= as.Date("2010-07-31"))

# Create msts (Multi-Seasonal Time Series) objects for training and validation
# Apply weekly (7) and yearly (365) seasonal periods
train_ts <- msts(train_data$daily_avg, seasonal.periods = c(7, 365), start = c(2005, 1))
valid_ts <- msts(valid_data$daily_avg, seasonal.periods = c(7, 365), start = c(2010, 182))
# 182 represents the approximate day of the year for July 1st
```

3. Top 5 Models: Analysis and Model Selection

In the comprehensive modeling phase of this project, we evaluated 19 distinct time series forecasting approaches, ranging from traditional statistical methods (TBATS, STLF-ARIMA) to machine learning techniques (NNAR, Prophet). Across all experiments, the Facebook Prophet framework consistently demonstrated superior predictive accuracy for the daily electricity load, ultimately securing all Top 5 positions (Please refer to the appendix for the score summary board and the complete code and performance evaluation of all models.)

Our baseline Prophet model (Model 8, Rank 7) provided a strong starting point, but we observed immediate improvements by incorporating deterministic calendar events, specifically US Holidays (Model 11, Rank 6). Since industrial and commercial electricity consumption drops significantly during holidays like Independence Day, allowing the model to account for these anomalies inherently reduced forecasting errors. From there, applying targeted hyperparameter tuning further optimized our predictions (Models 12, 13, and 14, tied for

Rank 3/4/5). Specifically, we increased the flexibility of the yearly seasonality (yearly.seasonality = 15) and adjusted the trend flexibility by setting changepoint.prior.scale = 0.1. This adjustment allowed the model to adapt more rapidly to recent trend shifts in the load data.

The most critical breakthrough in our modeling strategy was the transition from additive to multiplicative seasonality. In electricity demand, the amplitude of seasonal fluctuations (especially during peak summer cooling periods) tends to grow proportionally with the overall trend. By changing the seasonality mode to multiplicative (Model 16, Rank 2) and combining it with conditional weekly seasonality to separate summer and non-summer behavioral patterns, we achieved our best-performing model: Model 17 (Rank 1, Score: 0.0963).

Interestingly, our experiments revealed that incorporating external weather regressors (temperature and humidity forecasts) degraded model performance (e.g., Models 18 and 19). Because the weather variables for July 2011 had to be forecasted themselves, they introduced additional uncertainty and noise into the final load prediction. Consequently, our optimal strategy relied strictly on robust historical load patterns, explicit holiday effects, and dynamic seasonal structures without external weather dependencies.

APPENDIX. Forecast Models and Scores

Score Summary

Model Number	Model Description	Score on Kaggle	Rank
Model 1	TBATS	0.1021	
Model 2	STLF-ARIMA	0.1315	
Model 3	STLF + ETS	0.1330	
Model 4-1	ARIMA + Fourier terms (K = c(3,10))	0.1350	
Model 4-2	ARIMA + Fourier terms (K = c(3,15))	1.4840	
Model 4-3	ARIMA + Fourier terms (K = c(2,5))	0.1469	
Model 5	NNAR	0.1770	
Model 6	Ensemble Model (Averaging best outputs)	0.1211	
Model 7	ARIMAX with temperature forecast	0.2172	
Model 8	Prophet	0.0973	
Model 9	Prophet + Temp data	0.1052	
Model 10	Prophet + ARIMAX	0.1488	
Model 11	Prophet + US Holiday	0.0971	
Model 12	Prophet + US Holiday + Parameter tuning	0.0966	**TOP 3/4/5**
Model 13	Weighted Ensemble (Tuned Prophet + Base Prophet)	0.0966	**TOP 3/4/5**
Model 14	Prophet + US Holiday + Conditional Seasonality	0.0966	**TOP 3/4/5**
Model 15	NNAR + Temperature	0.1287	
Model 16	Prophet + US Holiday + Multiplicative Seasonality	0.0964	**TOP 2**
Model 17	Prophet + US Holiday + Multiplicative + Conditional Seasonality	**0.0963**	**TOP 1**

Model Number	Model Description	Score on Kaggle	Rank
Model 18	Prophet + Holiday + Multiplicative + Temp + Humid	0.1057	
Model 19	Prophet + Holiday + Multi + Cond + Temp + Humid	0.1063	

Model 12: Prophet + US Holiday + Parameter tuning

Score on Kaggle: 0.0966

```
# 1. Initialize Prophet with custom hyperparameters
# Increased yearly seasonality flexibility (15 instead of default 10)
# changepoint.prior.scale = 0.1 allows the trend to adapt faster
m_tuned <- prophet(yearly.seasonality = 15,
                   weekly.seasonality = TRUE,
                   daily.seasonality = FALSE,
                   changepoint.prior.scale = 0.1)

# Add US holidays
m_tuned <- add_country_holidays(m_tuned, country_name = 'US')

# 2. Fit and Predict
df_prophet_holidays <- data.frame(
  ds = daily_data$date,
  y = daily_data$daily_avg
)

m_tuned <- fit.prophet(m_tuned, df_prophet_holidays)
future_tuned <- make_future_dataframe(m_tuned, periods = 31)
forecast_tuned <- predict(m_tuned, future_tuned)

# 3. Extract and Save
prophet_tuned_preds <- tail(forecast_tuned$yhat, 31)
```

Model 13: Weighted Ensemble (Tuned Prophet + Base Prophet)

Score on Kaggle: 0.0966

```
### Model 11 used in Model 13
# 1. Initialize Prophet
m_holidays <- prophet(yearly.seasonality = TRUE,
                      weekly.seasonality = TRUE,
                      daily.seasonality = FALSE)

# Add US country holidays (e.g., Independence Day on July 4th)
m_holidays <- add_country_holidays(m_holidays, country_name = 'US')
m_holidays <- fit.prophet(m_holidays, df_prophet_holidays)


# 2. Create future dataframe for July 2011 (31 days)
future_dates_holidays <- make_future_dataframe(m_holidays, periods = 31)
```

```r
# 3. Generate the final forecast
print("Forecasting with Holidays...")
```

```
## [1] "Forecasting with Holidays..."
```

```r
forecast_prophet_holidays <- predict(m_holidays, future_dates_holidays)

# 4. Extract predictions
prophet_holidays_predictions <- tail(forecast_prophet_holidays$yhat, 31)
###

# Model A (Rank 1): Tuned Prophet + Holidays (70% weight)
weight_best <- 0.70

# Model B (Rank 2): Base Prophet + Holidays (30% weight)
weight_second <- 0.30

# Calculate the weighted average of the predictions
weighted_ensemble_preds <- (weight_best * prophet_tuned_preds) +
                           (weight_second * prophet_holidays_predictions)
```

Model 14: Prophet + US Holiday + Conditional Seasonality

Score on Kaggle: 0.0966

```r
library(lubridate)

# 1. Prepare data and add conditional columns for Summer vs. Other seasons
df_prophet_ultimate <- data.frame(
  ds = daily_data$date,
  y = daily_data$daily_avg
)

# Define 'Summer' as June, July, August, and September
df_prophet_ultimate$is_summer <- month(df_prophet_ultimate$ds) %in% c(6, 7, 8, 9)
df_prophet_ultimate$is_other <- !df_prophet_ultimate$is_summer

# 2. Initialize the Prophet model with Tuned Hyperparameters
# Turn OFF default weekly seasonality because we will define custom conditional ones.
m_ultimate <- prophet(yearly.seasonality = 15,          # Tuned parameter
                      weekly.seasonality = FALSE,       # Disabled for custom setup
                      daily.seasonality = FALSE,
                      changepoint.prior.scale = 0.1)    # Tuned parameter

# Add US country holidays
m_ultimate <- add_country_holidays(m_ultimate, country_name = 'US')

# 3. Add Custom Conditional Seasonalities
# This allows the model to learn a different weekly pattern for summer vs other months.
m_ultimate <- add_seasonality(m_ultimate, name = 'summer_weekly', period = 7,
                              fourier.order = 3, condition.name = 'is_summer')
m_ultimate <- add_seasonality(m_ultimate, name = 'other_weekly', period = 7,
                              fourier.order = 3, condition.name = 'is_other')
m_ultimate <- fit.prophet(m_ultimate, df_prophet_ultimate)
```

```r
# 4. Create future dataframe for July 2011 (31 days) and apply conditions
future_ultimate <- make_future_dataframe(m_ultimate, periods = 31)
future_ultimate$is_summer <- month(future_ultimate$ds) %in% c(6, 7, 8, 9)
future_ultimate$is_other <- !future_ultimate$is_summer

# 5. Generate the final forecast
print("Forecasting with Ultimate Prophet...")
```

```
## [1] "Forecasting with Ultimate Prophet..."
```

```r
forecast_ultimate <- predict(m_ultimate, future_ultimate)

# 6. Extract predictions and save to CSV
prophet_ultimate_preds <- tail(forecast_ultimate$yhat, 31)
```

Model 16: Prophet + US Holiday + Multiplicative Seasonality

Score on Kaggle: 0.0964

```r
# 1. Prepare data (reusing df_prophet_holidays from Model 11/12)
df_prophet_multi <- data.frame(
  ds = daily_data$date,
  y = daily_data$daily_avg
)

# 2. Initialize Prophet with multiplicative seasonality and same hyperparameters from Model 12
m_multi <- prophet(
  yearly.seasonality = 15,
  weekly.seasonality = TRUE,
  daily.seasonality = FALSE,
  seasonality.mode = 'multiplicative',
  changepoint.prior.scale = 0.1
)

# Add US holidays
m_multi <- add_country_holidays(m_multi, country_name = 'US')

# 3. Fit and Predict
m_multi <- fit.prophet(m_multi, df_prophet_multi)
future_multi <- make_future_dataframe(m_multi, periods = 31)
forecast_multi <- predict(m_multi, future_multi)

# 4. Extract and Save
prophet_multi_preds <- tail(forecast_multi$yhat, 31)
```

Model 17: Prophet + US Holiday + Multiplicative Seasonality + Conditional Seasonality

Score on Kaggle: 0.0963

```r
# 1. Prepare data with summer/other condition flags
df_prophet_multi_cond <- data.frame(
  ds = daily_data$date,
  y = daily_data$daily_avg
```

```r
)
df_prophet_multi_cond$is_summer <- month(df_prophet_multi_cond$ds) %in% c(6, 7, 8, 9)
df_prophet_multi_cond$is_other <- !df_prophet_multi_cond$is_summer

# 2. Initialize Prophet with multiplicative seasonality + tuned params + no default weekly
m_multi_cond <- prophet(
  yearly.seasonality = 15,
  weekly.seasonality = FALSE,
  daily.seasonality = FALSE,
  seasonality.mode = 'multiplicative',
  changepoint.prior.scale = 0.1
)

# Add US holidays
m_multi_cond <- add_country_holidays(m_multi_cond, country_name = 'US')

# 3. Add conditional weekly seasonalities (summer vs other)
m_multi_cond <- add_seasonality(m_multi_cond, name = 'summer_weekly', period = 7,
                                fourier.order = 3, condition.name = 'is_summer')
m_multi_cond <- add_seasonality(m_multi_cond, name = 'other_weekly',  period = 7,
                                fourier.order = 3, condition.name = 'is_other')

# 4. Fit and Predict
m_multi_cond <- fit.prophet(m_multi_cond, df_prophet_multi_cond)

future_multi_cond <- make_future_dataframe(m_multi_cond, periods = 31)
future_multi_cond$is_summer <- month(future_multi_cond$ds) %in% c(6, 7, 8, 9)
future_multi_cond$is_other <- !future_multi_cond$is_summer

forecast_multi_cond <- predict(m_multi_cond, future_multi_cond)

# 5. Extract and Save
prophet_multi_cond_preds <- tail(forecast_multi_cond$yhat, 31)
```